

[flume] AI 辅助组件选择和工作流脚本编写

课程辅助资料

此处为语雀内容卡片，点击链接查看：

<https://www.yuque.com/u27647325/giqki7/wazm0v4r0pk1fzse>

练习题

习题一 编写 agent 配置文件

请使用**极简数据生成器**向一个文件中写入数据，然后使用 GPT 编写 agent 配置文件，使用 `taildir source` 将数据同步到 hdfs 上。

习题二 编写 Flume 启停脚本

flume 自带的 `flume-ng` 命令只有启动功能，没有停止功能。因此通常需要我们自己编写 bash 脚本来管理 flume 同步任务的启动和停止。现在，请用 GPT 来完成启停脚本的开发。

习题三 编写时间戳拦截器

我们将要用 Flume 采集的数据格式如下：`{ "user_id": int, "event": string, "ts": long }` 其中 `ts` 是毫秒时间戳，现在请使用 GPT 编写一个时间戳拦截器。

习题四 修改堆内存溢出异常

在默认的 flume 配置下，当采集的数据量在速度和数量上变多时，有可能会发生下面的异常

```
java.lang.OutOfMemoryError: GC overhead limit exceeded
```

这个时候，我们也可以向 GPT 询问问题的解决方案。

参考答案

习题一 编写 agent 配置文件

第一轮提示

提示

我想在使用 flume 来将 linux 本地目录里的数据同步到 hdfs 上

请帮我写一个 agent 的配置来完成这项功能

1. 使用 taildir source 同步/opt/module/gpt/data 下的数据
2. 使用 file channel
3. 写到 hdfs 的/data/gpt 目录下

输出的文件请使用 gzip 格式压缩

答

为了实现这个需求，你可以参考以下 Flume agent 的配置：

```
# 定义 agent 名称
agent1.sources = taildirSource
agent1.channels = fileChannel
agent1.sinks = hdfsSink

# 配置 taildir source
agent1.sources.taildirSource.type = TAILDIR
agent1.sources.taildirSource.channels = fileChannel
agent1.sources.taildirSource.filegroups = f1
agent1.sources.taildirSource.filegroups.f1 = /opt/module/gpt/data/*.
agent1.sources.taildirSource.positionFile = /opt/module/gpt/taildir_position.json
```

```

# 配置 file channel
agent1.channels.fileChannel.type = file
agent1.channels.fileChannel.checkpointDir = /opt/module/gpt/checkpoint
agent1.channels.fileChannel.dataDirs = /opt/module/gpt/data-channel

# 配置 HDFS sink
agent1.sinks.hdfsSink.type = hdfs
agent1.sinks.hdfsSink.channel = fileChannel
agent1.sinks.hdfsSink.hdfs.path = hdfs://namenode:port/data/gpt
agent1.sinks.hdfsSink.hdfs.fileType = CompressedStream
agent1.sinks.hdfsSink.hdfs.codeC = gzip
agent1.sinks.hdfsSink.hdfs.batchSize = 1000
agent1.sinks.hdfsSink.hdfs.rollSize = 0
agent1.sinks.hdfsSink.hdfs.rollCount = 10000
agent1.sinks.hdfsSink.hdfs.rollInterval = 600

# 启动 agent
agent1.sources.taildirSource.selector.type = replicating
agent1.sources.taildirSource.selector.optionalChannels = fileChannel

```

请确保已经安装了 Flume 并将其配置为 classpath。然后将上述配置保存为一个名为 **flume-agent.conf** 的文件，并使用以下命令启动 Flume agent：

```

flume-ng agent --conf /path/to/flume/conf --conf-file /path/to/flume-agent.conf --name
agent1 -Dflume.root.logger=INFO,console

```

这个配置会创建一个 Flume agent，使用 taildir source 监控 **/opt/module/gpt/data** 目录下的文件变化，将数据写入 file channel，并最终将数据以 gzip 格式压缩后写入 HDFS 的 **/data/gpt** 目录下。此外，我们将 **taildir_position.json** 文件放在与数据文件分开的目录中，以避免将其误采集。

反馈

现在，我们可以将这个配置文件拿下来进行测试。可以看到数据已经能够成功采集到 hdfs 上

☐	Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name	☐
☐	-rw-r--r--	atguigu	supergroup	62.4 KB	Apr 17 13:57	3	128 MB	FlumeData.1681710719169.gz	☐
☐	-rw-r--r--	atguigu	supergroup	56.4 KB	Apr 17 13:57	3	128 MB	FlumeData.1681710719170.gz	☐
☐	-rw-r--r--	atguigu	supergroup	56.55 KB	Apr 17 13:57	3	128 MB	FlumeData.1681710719171.gz	☐
☐	-rw-r--r--	atguigu	supergroup	56.24 KB	Apr 17 13:58	3	128 MB	FlumeData.1681710719172.gz	☐
☐	-rw-r--r--	atguigu	supergroup	1.77 KB	Apr 17 13:58	3	128 MB	FlumeData.1681710719173.gz.tmp	☐

Showing 1 to 5 of 5 entries

Previous 1 Next

你还可以根据实际的运行效果继续迭代这个配置文件。

习题二 编写 Flume 启停脚本

第一轮提示

提示

请帮我写一个 base shell 脚本来管理 flume 任务的启停

我的 flume 安装路径在 /opt/module/flume

1. 这个脚本有三个子命令 start stop 和 help
2. 使用 start 命令时，脚本需要传递两个参数。
 1. 第一个参数是 flume-agent 的配置文件路径
 2. 第二个参数是 flume-agent 配置文件中 agent 的名称
3. 使用 stop 命令时，脚本需要传递一个参数，这个参数是 agent 的名称
4. 使用 help 命令时，打印这个脚本的用法

答

你可以使用以下脚本来管理 Flume 任务的启停：

```
#!/bin/bash

FLUME_HOME="/opt/module/flume"

function start_agent() {
    local config_file="$1"
    local agent_name="$2"

    if [ -z "$config_file" ] || [ -z "$agent_name" ]; then
        echo "错误：启动代理时需要提供配置文件路径和代理名称"
        exit 1
    fi

    echo "正在启动 Flume 代理：$agent_name"
    nohup $FLUME_HOME/bin/flume-ng agent --conf $FLUME_HOME/conf --conf-file
"$config_file" --name "$agent_name" >/dev/null 2>&1 &
}

function stop_agent() {
    local agent_name="$1"

    if [ -z "$agent_name" ]; then
        echo "错误：停止代理时需要提供代理名称"
        exit 1
    fi

    local pid=$(ps -ef | grep "org.apache.flume.node.Application" | grep "$agent_name" |
awk '{print $2}')
    if [ -z "$pid" ]; then
        echo "未找到代理：$agent_name"
    else
        echo "正在停止 Flume 代理：$agent_name (pid: $pid)"
    fi
}
```

```
        kill "$pid"
    fi
}

function print_help() {
    echo "用法: "
    echo "  flume_manage.sh start <配置文件路径> <代理名称> - 启动指定的 Flume 代理"
    echo "  flume_manage.sh stop <代理名称> - 停止指定的 Flume 代理"
    echo "  flume_manage.sh help - 显示帮助信息"
}

if [ $# -eq 0 ]; then
    print_help
    exit 1
fi

case "$1" in
    start)
        start_agent "$2" "$3"
        ;;
    stop)
        stop_agent "$2"
        ;;
    help)
        print_help
        ;;
    *)
        echo "未知命令: $1"
        print_help
        exit 1
        ;;
esac
```

将这个脚本保存为 `flume_manage.sh`，并给予执行权限（`chmod +x flume_manage.sh`）。然后你就可以使用这个脚本来管理 Flume 任务的启停了。

这个脚本有三个子命令：`start`、`stop` 和 `help`。使用 `start` 命令时，需要传递两个参数：配置文件路径和代理名称。使用 `stop` 命令时，需要传递一个参数：代理名称。使用 `help` 命令时，脚本将打印用法。

反馈

我们应当在测试环境里对这个脚本的功能做一些测试。

```
正在启动 flume 代理: agent1
[atguigu@hadoop102 gpt]$ ps -ef | grep flume
atguigu  93633      1 99 15:20 pts/1    00:00:12 /opt/module/java/bin/java -Xmx20m -cp /opt/module/hadoop/share/hadoop/common/*:/opt/module/hadoop/share/hadoop/hdfs:/opt/module/hadoop/share/hadoop/mapreduce/*:/opt/module/hadoop/share/hadoop/yarn:/opt/module/hadoop/share/hadoop/yarn/lib/*:flume.node.Application --conf-file /opt/module/flume/job/gpt_to_hdfs.conf --name agent1
atguigu  93825  56916  0 15:20 pts/1    00:00:00 grep --color=auto flume
[atguigu@hadoop102 gpt]$ ./flume_manage.sh stop agent1
正在停止Flume代理: agent1 (pid: 93633)
[atguigu@hadoop102 gpt]$ ps -ef | grep flume
atguigu  93984  56916  0 15:21 pts/1    00:00:00 grep --color=auto flume
```

最后发现这个脚本完全符合我们的预期。可以考虑上线。

习题三 编写时间戳拦截器

第一轮提示

提示

帮我编写一个 Flume 的时间戳拦截器

我的 json 格式如下

```
{

  "user_id": int,

  "event": string,

  "ts": long
```

```
}
```

其中 `ts` 是毫秒时间戳，帮我把它取出来放到 flume event 的 header 里。

答

要实现这个需求，您需要创建一个自定义的 Flume 拦截器。首先，需要创建一个 Java 类，实现 **Interceptor** 接口。然后，解析 JSON 格式的数据，从中提取 **ts** 字段，并将其作为 Flume 事件的 header。

以下是一个简单的示例：

```
import com.google.gson.Gson;
import org.apache.flume.Context;
import org.apache.flume.Event;
import org.apache.flume.interceptor.Interceptor;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;

public class TimestampInterceptor implements Interceptor {
    private final Gson gson;

    public TimestampInterceptor() {
        gson = new Gson();
    }

    @Override
    public void initialize() {}

    @Override
    public Event intercept(Event event) {
        String body = new String(event.getBody());
        Map<String, Object> jsonData = gson.fromJson(body, Map.class);

        if (jsonData.containsKey("ts")) {
```



```

        long timestamp = ((Number) jsonData.get("ts")).longValue();
        event.getHeaders().put("timestamp", Long.toString(timestamp));
    }
    return event;
}

@Override
public List<Event> intercept(List<Event> events) {
    List<Event> interceptedEvents = new ArrayList<>(events.size());
    for (Event event : events) {
        Event interceptedEvent = intercept(event);
        if (interceptedEvent != null) {
            interceptedEvents.add(interceptedEvent);
        }
    }
    return interceptedEvents;
}

@Override
public void close() {}

public static class Builder implements Interceptor.Builder {
    @Override
    public Interceptor build() {
        return new TimestampInterceptor();
    }

    @Override
    public void configure(Context context) {}
}
}

```

将该代码保存为 **TimestampInterceptor.java** 文件，然后编译并打包为 JAR 文件。将 JAR 文件放到 Flume 的 **lib** 目录下。

接下来，在 Flume 配置文件中，配置拦截器。假设您的 source 是一个名为 **mySource** 的 HTTP Source：

```
mySource.sources.http-source.interceptors = timestampInterceptor
mySource.sources.http-source.interceptors.timestampInterceptor.type =
com.example.TimestampInterceptor$Builder
```

反馈

这个程序为我们提供了，一个时间戳拦截器的实现代码。从逻辑上看这段代码是正确的。感兴趣的同学可以自己编辑部署测试效果。

习题四 修改堆内存溢出异常

第一轮提示

提示

我用 flume 将日志文件采集到 hdfs 上，

报错 `java.lang.OutOfMemoryError: GC overhead limit exceeded`

该如何处理

答

`java.lang.OutOfMemoryError: GC overhead limit exceeded` 错误表明 Flume 在处理数据时遇到了内存不足的问题。这可能是因为 Flume 的堆内存设置太小，或者某些配置导致了内存泄漏。以下是一些建议来解决这个问题：

1. 增加 Flume 的堆内存分配： 你可以通过调整 Flume 的 JVM 参数来增加堆内存分配。在启动 Flume 时，设置 `FLUME_JAVA_OPTS` 环境变量来指定 JVM 参数。例如，你可以将堆内存设置为 2GB：

```
export FLUME_JAVA_OPTS="-Xms2g -Xmx2g"
```

2. 或者你可以将这个环境变量添加到 `flume-env.sh` 文件中。这个文件通常位于 Flume 的 `conf` 目录下。确保你增加的内存大小适用于你的机器。
3. 调整 Flume 的配置： 如果增加堆内存分配后仍然出现内存问题，你可能需要调整 Flume 的配置。以下是一些建议：
 - 减少批处理大小：降低 `batchSize` 参数，以减少单次处理的事件数量。
 - 减少通道容量：降低通道的 `capacity` 参数，以减少缓冲区中的事件数量。
 - 调整 Source、Channel 和 Sink 的类型：尝试使用不同的组件类型，以找到最适合你的场景的解决方案。
4. 检查 Flume 插件和自定义组件： 如果你正在使用第三方插件或自定义组件，请确保它们没有内存泄漏的问题。如果有疑虑，尝试暂时禁用这些组件，看看是否仍然出现内存问题。
5. 分析堆内存快照： 如果上述方法都不能解决问题，你可能需要深入了解内存泄漏的根本原因。可以使用诸如 VisualVM、MAT (Memory Analyzer Tool) 或 jhat 等工具分析 Flume 的堆内存快照。这将帮助你找到内存泄漏的具体位置，以便进行修复。

通过以上建议，你应该能够解决 Flume 遇到的 `java.lang.OutOfMemoryError: GC overhead limit exceeded` 错误。在调整配置和内存分配时，请密切关注 Flume 的性能和资源使用情况，以确保你的修改不会对其他部分产生负面影响。

反馈

这个方案很准，通常我们都是使用第一个方案（增加 jvm 的堆内存）来解决问题。